10.1-8
Show how to implement a stack using two queues. Analyze the running time of the stack operations.

10.2 Linked lists

A ***linked list*** is a data structure in which the objects are arranged in a linear order. Unlike an array, however, in which the linear order is determined by the array indices, the order in a linked list is determined by a pointer in each object. Since the elements of linked lists often contain keys that can be searched for, linked lists are sometimes called ***search lists***. Linked lists provide a simple, flexible representation for dynamic sets, supporting (though not necessarily efficiently) all the operations listed on page 250.

As shown in Figure 10.4, each element of a ***doubly linked list*** L is an object with an attribute *key* and two pointer attributes: *next* and *prev*. The object may also contain other satellite data. Given an element x in the list, $x.next$ points to its successor in the linked list, and $x.prev$ points to its predecessor. If $x.prev = \text{NIL}$, the element x has no predecessor and is therefore the first element, or ***head***, of the list. If $x.next = \text{NIL}$, the element x has no successor and is therefore the last element, or ***tail***, of the list. An attribute $L.head$ points to the first element of the list. If $L.head = \text{NIL}$, the list is empty.

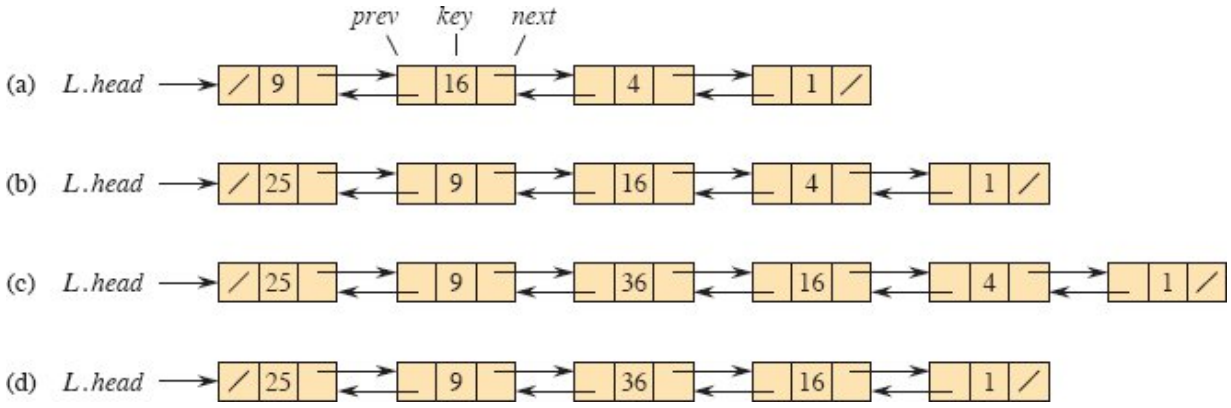

Figure 10.4 (a) A doubly linked list L representing the dynamic set $\{1, 4, 9, 16\}$. Each element in the list is an object with attributes for the key and pointers (shown by arrows) to the next and previous objects. The *next* attribute of the tail and the *prev* attribute of the head are NIL, indicated by a diagonal slash. The attribute $L.head$ points to the head. **(b)** Following the execution of LIST-PREPEND(L, x), where $x.key = 25$, the linked list has an object with key 25 as the new head. This new object points to the old head with key 9. **(c)** The result of calling LIST-INSERT(x, y), where $x.key = 36$ and y points to the object with key 9. **(d)** The result of the subsequent call LIST-DELETE(L, x), where x points to the object with key 4.

A list may have one of several forms. It may be either singly linked or doubly linked, it may be sorted or not, and it may be circular or not. If a list is ***singly linked***, each element has a *next* pointer but not a *prev* pointer. If a list is ***sorted***, the linear order of the list corresponds to the linear order of keys stored in elements of the list. The minimum element is then the head of the list, and the maximum element is the tail. If the list is ***unsorted***, the elements can appear in any order. In a ***circular list***, the *prev* pointer of the head of the list points to the tail, and the *next* pointer of the tail of the list points to the head. You can think of a circular list as a ring of elements. In the remainder of this section, we assume that the lists we are working with are unsorted and doubly linked.

Searching a linked list

The procedure LIST-SEARCH(L, k) finds the first element with key k in list L by a simple linear search, returning a pointer to this element. If no object with key k appears in the list, then the procedure returns NIL. For the linked list in Figure 10.4(a), the call LIST-SEARCH(L, 4)

returns a pointer to the third element, and the call LIST-SEARCH(L, 7) returns NIL. To search a list of n objects, the LIST-SEARCH procedure takes $\Theta(n)$ time in the worst case, since it may have to search the entire list.

```
LIST-SEARCH(L, k)

1 x = L.head
2 while x ≠ NIL and x.key ≠ k
3     x = x.next
4 return x
```

Inserting into a linked list

Given an element x whose *key* attribute has already been set, the LIST-PREPEND procedure adds x to the front of the linked list, as shown in Figure 10.4(b). (Recall that our attribute notation can cascade, so that *L.head.prev* denotes the *prev* attribute of the object that *L.head* points to.) The running time for LIST-PREPEND on a list of n elements is $O(1)$.

```
LIST-PREPEND(L, x)

1 x.next = L.head
2 x.prev = NIL
3 if L.head ≠ NIL
4     L.head.prev = x
5 L.head = x
```

You can insert anywhere within a linked list. As Figure 10.4(c) shows, if you have a pointer y to an object in the list, the LIST-INSERT procedure on the facing page "splices" a new element x into the list, immediately following y, in $O(1)$ time. Since LIST-INSERT never references the list object L, it is not supplied as a parameter.

```
LIST-INSERT(x, y)
```

```
1 x.next = y.next
2 x.prev = y
3 if y.next ≠ NIL
4       y.next.prev = x
5 y.next = x
```

Deleting from a linked list

The procedure LIST-DELETE removes an element x from a linked list L. It must be given a pointer to x, and it then "splices" x out of the list by updating pointers. To delete an element with a given key, first call LIST-SEARCH to retrieve a pointer to the element. Figure 10.4(d) shows how an element is deleted from a linked list. LIST-DELETE runs in $O(1)$ time, but to delete an element with a given key, the call to LIST-SEARCH makes the worst-case running time be $\Theta(n)$.

```
LIST-DELETE(L, x)

1 if x.prev ≠ NIL
2       x.prev.next = x.next
3 else L.head = x.next
4 if x.next ≠ NIL
5       x.next.prev = x.prev
```

Insertion and deletion are faster operations on doubly linked lists than on arrays. If you want to insert a new first element into an array or delete the first element in an array, maintaining the relative order of all the existing elements, then each of the existing elements needs to be moved by one position. In the worst case, therefore, insertion and deletion take $\Theta(n)$ time in an array, compared with $O(1)$ time for a doubly linked list. (Exercise 10.2-1 asks you to show that deleting an element from a singly linked list takes $\Theta(n)$ time in the worst case.) If, however, you want to find the kth element in the linear order, it takes just $O(1)$ time in an array regardless of k, but in a linked list, you'd have to traverse k elements, taking $\Theta(k)$ time.

Sentinels

The code for LIST-DELETE is simpler if you ignore the boundary conditions at the head and tail of the list:

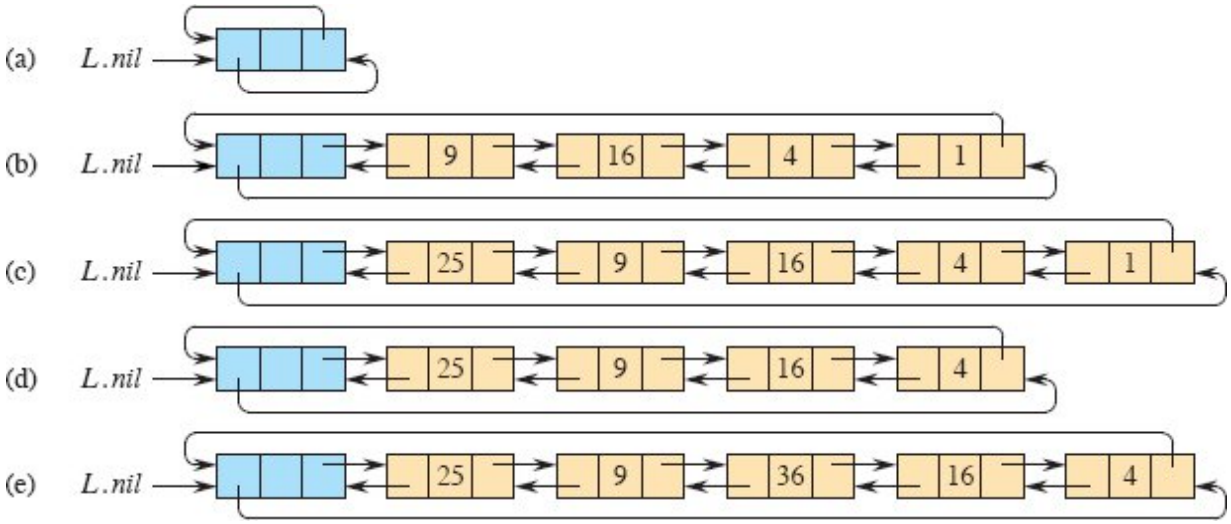

Figure 10.5 A circular, doubly linked list with a sentinel. The sentinel *L.nil*, in blue, appears between the head and tail. The attribute *L.head* is no longer needed, since the head of the list is *L.nil.next*. **(a)** An empty list. **(b)** The linked list from Figure 10.4(a), with key 9 at the head and key 1 at the tail. **(c)** The list after executing LIST-INSERT′ (x, *L.nil*), where $x.key = 25$. The new object becomes the head of the list. **(d)** The list after deleting the object with key 1. The new tail is the object with key 4. **(e)** The list after executing LIST-INSERT′ (x, y), where $x.key = 36$ and y points to the object with key 9.

```
LIST-DELETE′ (x)
1 x.prev.next = x.next
2 x.next.prev = x.prev
```

A ***sentinel*** is a dummy object that allows us to simplify boundary conditions. In a linked list L, the sentinel is an object *L.nil* that represents NIL but has all the attributes of the other objects in the list. References to NIL are replaced by references to the sentinel *L.nil*. As shown in Figure 10.5, this change turns a regular doubly linked list into a ***circular, doubly linked list with a sentinel***, in which the sentinel *L.nil* lies between the head and tail. The attribute *L.nil.next* points to the head of the list, and *L.nil.prev* points to the tail. Similarly, both the *next*

attribute of the tail and the *prev* attribute of the head point to *L.nil*. Since *L.nil.next* points to the head, the attribute *L.head* is eliminated altogether, with references to it replaced by references to *L.nil.next*. Figure 10.5(a) shows that an empty list consists of just the sentinel, and both *L.nil.next* and *L.nil.prev* point to *L.nil*.

To delete an element from the list, just use the two-line procedure LIST-DELETE′ from before. Just as LIST-INSERT never references the list object *L*, neither does LIST-DELETE′. You should never delete the sentinel *L.nil* unless you are deleting the entire list!

The LIST-INSERT′ procedure inserts an element x into the list following object y. No separate procedure for prepending is necessary: to insert at the head of the list, let y be *L.nil*; and to insert at the tail, let y be *L.nil.prev*. Figure 10.5 shows the effects of LIST-INSERT′ and LIST-DELETE′ on a sample list.

```
LIST-INSERT′ (x, y)
1 x.next = y.next
2 x.prev = y
3 y.next.prev = x
4 y.next = x
```

Searching a circular, doubly linked list with a sentinel has the same asymptotic running time as without a sentinel, but it is possible to decrease the constant factor. The test in line 2 of LIST-SEARCH makes two comparisons: one to check whether the search has run off the end of the list and, if not, one to check whether the key resides in the current element x. Suppose that you *know* that the key is somewhere in the list. Then you do not need to check whether the search runs off the end of the list, thereby eliminating one comparison in each iteration of the **while** loop.

The sentinel provides a place to put the key before starting the search. The search starts at the head *L.nil.next* of list *L*, and it stops if it finds the key somewhere in the list. Now the search is guaranteed to find the key, either in the sentinel or before reaching the sentinel. If the key is found before reaching the sentinel, then it really is in the element

where the search stops. If, however, the search goes through all the elements in the list and finds the key only in the sentinel, then the key is not really in the list, and the search returns NIL. The procedure LIST-SEARCH′ embodies this idea. (If your sentinel requires its *key* attribute to be NIL, then you might want to assign *L.nil.key* = NIL before line 5.)

```
LIST-SEARCH′ (L, k)
1 L.nil.key = k     // store the key in the sentinel to guarantee it is in list
2 x = L.nil.next    // start at the head of the list
3 while x.key ≠ k
4     x = x.next
5 if x == L.nil     // found k in the sentinel
6     return NIL    // k was not really in the list
7 else return x     // found k in element x
```

Sentinels often simplify code and, as in searching a linked list, they might speed up code by a small constant factor, but they don't typically improve the asymptotic running time. Use them judiciously. When there are many small lists, the extra storage used by their sentinels can represent significant wasted memory. In this book, we use sentinels only when they significantly simplify the code.

Exercises

10.2-1

Explain why the dynamic-set operation INSERT on a singly linked list can be implemented in $O(1)$ time, but the worst-case time for DELETE is $\Theta(n)$.

10.2-2

Implement a stack using a singly linked list. The operations PUSH and POP should still take $O(1)$ time. Do you need to add any attributes to the list?

10.2-3

Implement a queue using a singly linked list. The operations ENQUEUE and DEQUEUE should still take $O(1)$ time. Do you need to add any attributes to the list?

10.2-4
The dynamic-set operation UNION takes two disjoint sets S_1 and S_2 as input, and it returns a set $S = S_1 \cup S_2$ consisting of all the elements of S_1 and S_2. The sets S_1 and S_2 are usually destroyed by the operation. Show how to support UNION in $O(1)$ time using a suitable list data structure.

10.2-5
Give a $\Theta(n)$-time nonrecursive procedure that reverses a singly linked list of n elements. The procedure should use no more than constant storage beyond that needed for the list itself.

★ ***10.2-6***
Explain how to implement doubly linked lists using only one pointer value $x.np$ per item instead of the usual two (*next* and *prev*). Assume that all pointer values can be interpreted as k-bit integers, and define $x.np = x.next$ XOR $x.prev$, the k-bit "exclusive-or" of $x.next$ and $x.prev$. The value NIL is represented by 0. Be sure to describe what information you need to access the head of the list. Show how to implement the SEARCH, INSERT, and DELETE operations on such a list. Also show how to reverse such a list in $O(1)$ time.

10.3 Representing rooted trees

Linked lists work well for representing linear relationships, but not all relationships are linear. In this section, we look specifically at the problem of representing rooted trees by linked data structures. We first look at binary trees, and then we present a method for rooted trees in which nodes can have an arbitrary number of children.

We represent each node of a tree by an object. As with linked lists, we assume that each node contains a *key* attribute. The remaining